Unit 2

SOFTWARE PROCESS DEVELOPMENT MODELS

2.1. Software Process Models:

- Software Processes is a coherent set of activities for specifying, designing, implementing and testing software systems.
- A software process model is an abstract representation of a process that presents a description of a process from some particular perspective.

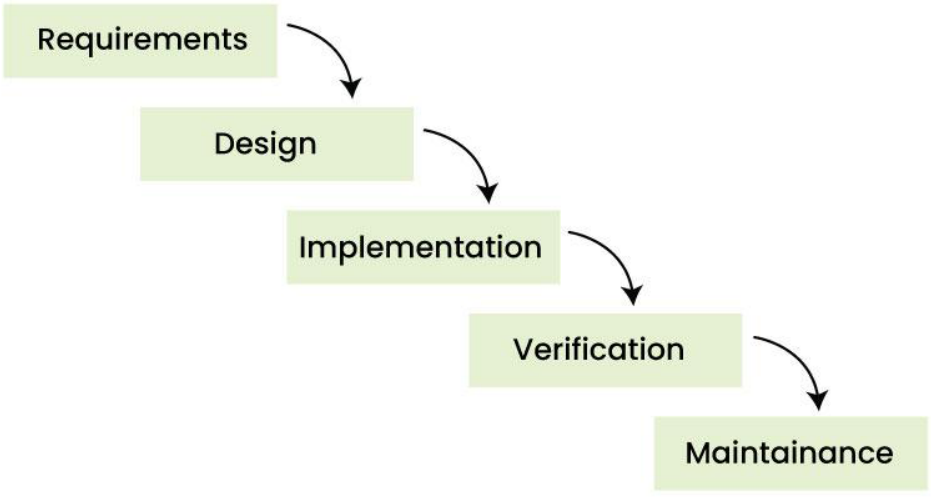
There are many different software processes but all involve:
Specification – defining what the system should do;
Design and implementation – defining the organization of the system and implementing the system;
Validation – checking that it does what the customer wants;
Evolution – changing the system in response to changing customer needs.

2.2. Software Development Life Cycle Models:

1. Waterfall Model:

- It is a famous and good version of SDLC for software engineering.
- The waterfall model is a linear and sequential model, which means that a development phase cannot begin until the previous phase is completed.
- We cannot overlap phases in waterfall model.
- We can imagine waterfall in the following way "Once the water starts flowing over the edge of the cliff, it starts falling down the mountain and the water cannot go back up."

Phases of Waterfall model:



Waterfall Model

Requirement phase: - Requirement phase is the first phase of the water Fall. In this phase the requirements of the system are collected and documented. This phase is very crucial because the next phases are based on this phase.

Design phase: - Design phase is based on the fact how the software will be built. The main objective of the design phase is to prepare the blueprint of the software system so that no problems are faced in the coming phases and solutions to all the requirements in the requirement phase are found.

Implementation phase: - In this phase, hardware, software and application programs are installed and the database design is implemented. Before the database design can be implemented, the software has to go through a testing, coding, and debugging process. This is the longest lasting phase in waterfall.

Verification phase: - In this phase the software is verified and it is evaluated that we have created the right product. In this phase, various types of testing are done and every area of the software is checked. It is believed that if we do not verify the software properly and there is any defect in it then no one will use it, hence verification is very important. One advantage of verification is that it reduces the risk of software failure.

Maintenance phase: - This is the last phase of waterfall. When the system is ready and users start using it, then the problems that arise have to be solved time-to-time. Taking care of the finished software and maintaining it as per time is called maintenance.

Advantages of Waterfall Model

- This model is simple and easy to understand.
- This is very useful for small projects.
- This model is easy to manage.
- The end goal is determined early.
- Each phase of this model is well explained.
- It provides a structured way to do things.

Disadvantages of Waterfall Model

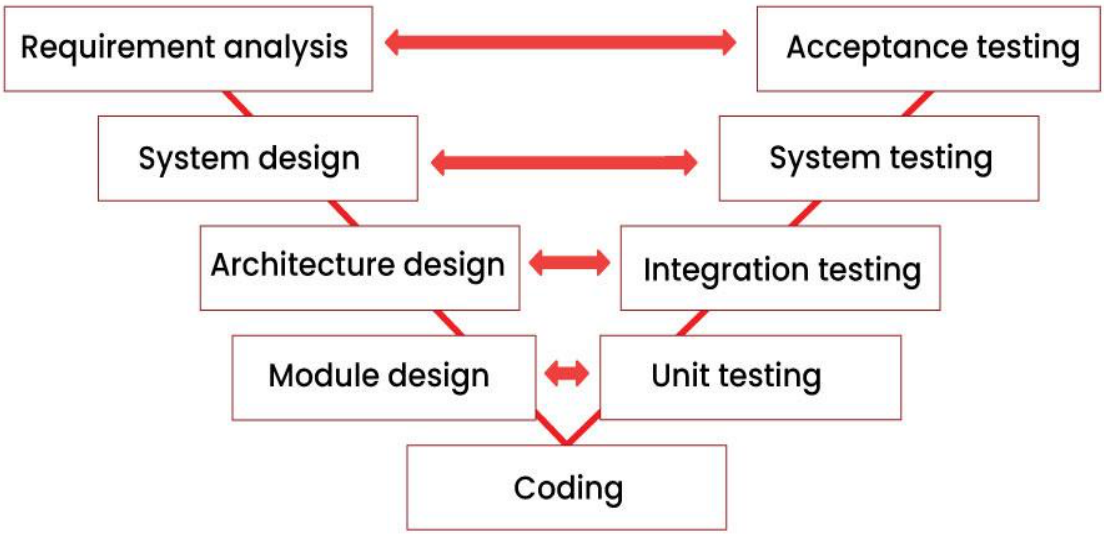
- In this model, complete and accurate requirements are expected at the beginning of the development process.
- Working software is not available for very long during the development life cycle.
- We cannot go back to the previous phase due to which it is very difficult to change the requirements.
- Risk is not assessed in this, hence there is high risk and uncertainty in this model.
- In this the testing period comes very late.
- Due to its sequential nature this model is not realistic in today's world.
- This is not a good model for large and complex projects.

2. **V-Model**

- V-Model is an SDLC model, it is also called Verification and Validation Model.
- V-Model is widely used in the Software Development Process, and it is considered a disciplined model.
- In V-Model, the execution of each process is sequential, that is, the new phase starts only after the previous phase ends.

- It is based on the association of testing phase with each development phase that is in V-Model with each development phase, its testing phase is also associated in a V-shape in other words both software development and testing activities take place at the same time.

Phases of V-model



Requirements analysis: - This is the first phase of the development cycle, in which the requirements of the product are analysed according to the customer's needs. In this phase, product related requirements are thoroughly collected from the customer. This is a very important phase because this phase determines the coming phases. In this phase, acceptance tests are designed for later use.

System design: - When we have the requirements of the product, after that we prepare a complete design of the system. In this phase, a complete description of the hardware and all the technical components required to develop the product.

Architectural design: - In this phase architectural specifications are designed. It contains the specification of how the software will link internally and externally with all the components. Therefore, this phase is also called high level design (HLD).

Module design: - In this phase the internal design of all the modules of the system is specified. Therefore, it is called low level design (LLD). It is very important that the design of all modules should be according to the system architecture. Unit tests are also designed in the module design phase.

Coding phase: - In the coding phase, coding of the design and specification done in the previous phases is done. This phase takes the most time.

Validation Phases of V-Model

Unit testing: - In the unit testing phase, the unit tests created during the module design phase are executed. Unit testing is code level testing, it only verifies the technical design. Therefore, it is not able to test all the defects.

Integration testing: - In integration testing, the integration tests created in the architectural design phase are executed. Integration testing ensures that all modules are working well together.

System testing: – In system testing, the system tests created in the system design phase are executed. System tests check the complete functionality of the system. In this, more attention is given to performance testing and regression testing.

Acceptance testing: - In acceptance testing, the acceptance tests created in the requirement analysis phase are executed. This testing ensures that the system is compatible with other systems. And in this, non-functional issues like: - load time, performance etc. are tested in the user environment.

Advantages of V-Model

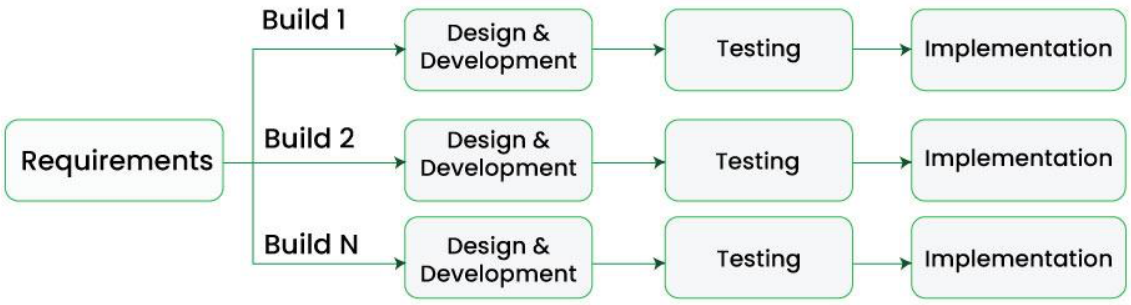
- This is a simple and easy to use model.
- Planning, testing and designing tests can be done even before coding.
- This is a very disciplined model, in which phase by phase development and testing goes on.
- Defects are detected in the initial stage itself.
- Small and medium scale developments can be easily completed using it.

Disadvantages of V-Model

- This model is not suitable for any complex projects.
- There remains both high risk and uncertainty.
- This is not a suitable model for an ongoing project.
- This model is not at all suitable for a project which is unclear and in which there are changes in the requirement.

3. **Incremental Model**

- In Incremental Model, the software development Process is divided into several increments and the same phases are followed in each increment.
- In simple language, under this model a complex project is developed in many modules or builds.
- For example, we collect the customer's requirements, now instead of making the entire software at once, we first take some requirements and based on them create a module or function of the software and deliver it to the customer. Then we take some more requirements and based on them add another module to that software.



Phases of Incremental Model

Communication: In the first phase, we talk face to face with the customer and collect his mandatory requirements. Like what functionalities does the customer want in his software, etc.

Planning: In this phase the requirements are divided into multiple modules and planning is done on their basis.
Modeling: In this phase the design of each module is prepared. After the design is ready, we take a particular module among many modules and save it in DDS (Design Document Specification). Diagrams like ERDs and DFDs are included in this document.
Construction: Here we start construction based on the design of that particular module. That is, the design of the module is implemented in coding. Once the code is written, it is tested.
Deployment: After the testing of the code is completed, if the module is working properly then it is given to the customer for use. After this, the next module is developed through the same phases and is combined with the previous module. This makes new functionality available to the customer. This will continue until complete modules are developed.

Advantages of Incremental Model

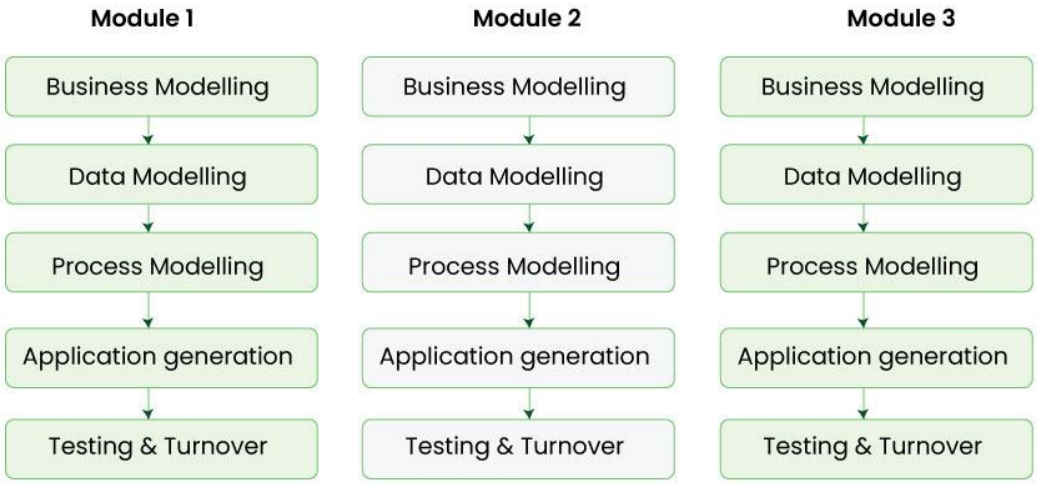
- This model is flexible and less expensive to change requirements and scope.
- The customer can respond to each module and provide feedback if any changes are needed.
- Project progress can be measured.
- It is easier to test and debug during a short iteration.
- Errors are easy to identify.

Disadvantages of Incremental Model

- Management is a continuous activity that must be handled.
- Before the project can be dismantled and built incrementally,
- The complete requirements of the software should be clear.
- This requires good planning and designing.
- The total cost of this model is higher.

4. **RAD Model**

- RAD model stands for rapid application development model.
- The methodology of RAD model is similar to that of incremental or waterfall model.
- It is used for small projects.
- If the project is large then it is divided into many small projects and these small projects are planned one by one and completed.
- In this way, by completing small projects, the large project gets ready quickly.



RAD Model

Advantage of RAD Model

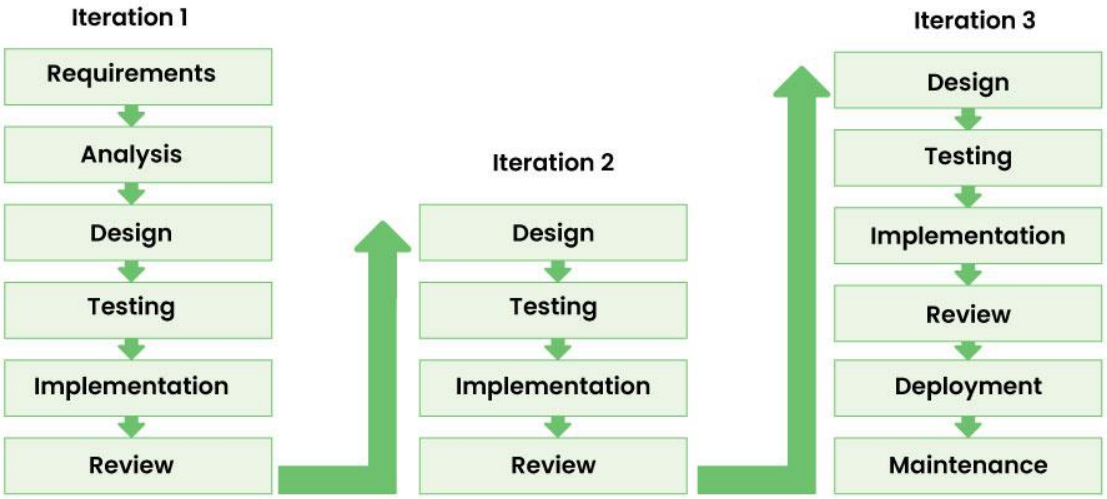
- It reduces the time taken in development.
- In this the components are reused.
- It is flexible and it is easy to make any changes in it.
- There are very few defects in it because it is a prototype by nature.
- In this, productivity can be increased in less time with less people.
- It is cost effective.
- It is suitable for small projects.

Disadvantages of RAD Model

- In this we need highly skilled developers and designers.
- It is very difficult to manage.
- It is not suitable for project that are complex and takes long time.
- In this, feedback from the client is required for the development of each phase.
- Automated code generation is very expensive.
- This model is suitable only for component based and scalable systems.

5. **Iterative Model**

- In Iterative model we start developing the software with some requirements and when it is developed, it is reviewed.
- If there are requirements for changes in it, then we develop a new version of the software based on those requirements.
- This process repeats itself many times until we get our final product.
- After the first version is developed, if there is a need to change the software, then a new version is developed with the second iteration.



Phases of iterative model

Requirement gathering & analysis: In this phase, all the software requirements of the customer are collected and it is analyzed whether those requirements can be meet or not. Besides, it is also checked whether this project will not go beyond our budget.

Design: In this phase the design of software is prepared. For this, various diagrams like Data Flow diagram, class diagram, activity diagram, state transition diagram, etc. are used.

Implementation: Now the design of software is implemented in coding through various programming languages. We also call this coding phase.

Testing: After the coding of the software is done, it is now tested so that the bugs and errors present in it can be identified. To do this, various testing techniques like performance testing, security testing, requirement testing, stress testing, etc. are done.

Deployment: Finally the software is given to the customer. After this the customer starts using that software in his work environment.

Review: After the software is deployed in its work environment, it is reviewed. If any error/bug is found or any new requirements come in front of developer, then again these phases are repeated with new iteration and a new version is developed.

Maintenance: In this phase we look at customer feedback, solve problems, fix errors, update software, etc.

Advantage of Iterative model

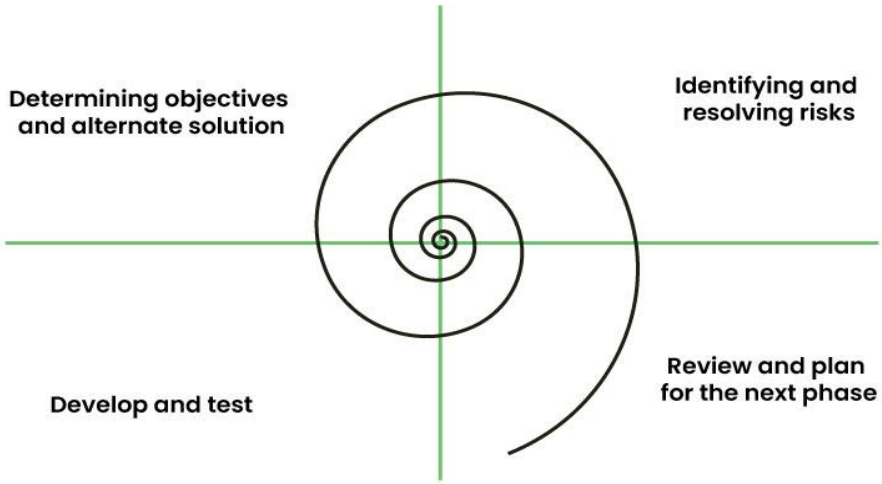
- In iterative models, bugs and errors can be identified quickly.
- Under this model, software is prepared quickly with some specifications.
- Testing and debugging the software becomes easier during each iteration.
- We get reliable feedback from users along with blueprints.
- This model is easily adaptable to constantly changing needs.
- Risks are identified and resolved during iteration.

Disadvantage of Iterative model:

- Iterative model is not suitable for small projects.
- Since the requirements are constantly changing, we have to make frequent changes in the software.
- Due to constantly changing requirements, the budget of the project also increases and it takes more time to complete it.
- In this model, it is complicated to control the entire process of software development.
- It is very difficult to tell by what date the complete software will be ready.

6. **Spiral Model**

- This model has characteristics of both iterative and waterfall models.
- This model is used in projects which are large and complex.
- This model was named spiral because if we look at its figure, it looks like a spiral, in which a long-curved line starts from the centre point and makes many loops around it.
- The number of loops in the spiral is not decided in advance but it depends on the size of the project and the changing requirements of the user.
- We also call each loop of the spiral a phase of the software development process.
- It was first developed by Barry Boehm in 1986.



Spiral model

These phases are as follows: -

Determining objectives and alternate solutions: In the first phase, whatever requirements the customer has related to the software are collected. On the basis of which objectives are identified and analysed and various alternative solutions are proposed.

Identifying and resolving risks: In this phase, all the proposed solutions are assessed and the best solution is selected. Now that solution is analysed and the risks related to it are identified. Now the identified risks are resolved through some best strategy.

Develop and test: Now the development of software is started. In this phase various features are implemented, that is, their coding is done. Then those features are verified through testing.

Review and plan for the next phase: In this phase the developed version of the software is given to the customer and he evaluates it. Gives his feedback and tells new requirements. Finally planning for the next phase (next spiral) is started.

Advantages of Spiral Model

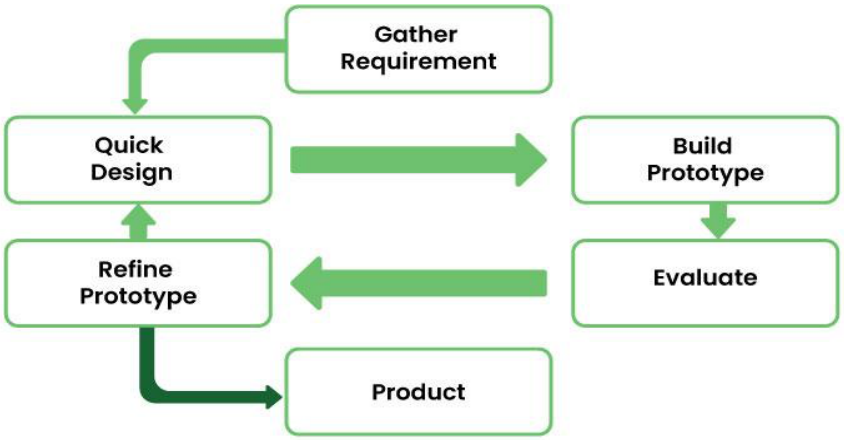
- If we have to add additional functionality or make any changes to the software, then through this model we can do so in the later stages also.
- Spiral model is suitable for large and complex projects.
- It is easy to estimate how much the project will cost.
- Risk analysis is done in each phase of this model.
- Since continuous feedback is taken from the customer during the development process, the chances of customer satisfaction increases.

Disadvantage of Spiral Model

- This is the most complex model of SDLC, due to which it is quite difficult to manage.
- This model is not suitable for small projects.
- The cost of this model is quite high.
- It requires more documentation than other models.
- Experienced experts are required to evaluate and review the project from time to time.
- Using this model, the success of the project depends greatly on the risk analysis phase.

7. **Prototype model**

- Prototype model is an activity in which prototypes of software applications are created.
- First a prototype is created and then the final product is manufactured based on that prototype.
- The prototype model was developed to overcome the shortcomings of the waterfall model.
- This model is created when we do not know the requirements well.
- The specialty of this model is that this model can be used with other models as well as alone.



Prototype model

The prototype model has the following phases

Requirement gathering: The first step of prototype model is to collect the requirements, although the customer does not know much about the requirements but the major requirements are defined in detail.

Build the initial prototype: In this phase the initial prototype is built. In this some basic requirements are displayed and user interface is made available.

Review the prototype: When the construction of the prototype is completed, it is presented to the end users or customer and feedback is taken from them about this prototype. This feedback is used to further improve the system and possible changes are made to the prototype.

Revise and improve the prototype: When feedback is taken from end users and customers, the prototype is improved on the basis of feedback. If the customer is not satisfied with the prototype, a new prototype is created and this process continues until the customer gets the prototype as per his desire.

Advantages of Prototype model

- Prototype Model is suggested to create applications whose prototype is very easy and which always includes human machine interaction within it.
- When we know only the general objective of creating software, but we do not know anything in detail about input, processing and output. Then in such a situation we make it a Prototype Model.
- When a software developer is not very sure about the capability of an algorithm or its adaptability to an operating system, then in this situation, using a prototype model can be a better option.

Disadvantages of Prototype model

- When the first version of the prototype model is ready, the customer himself often wants small fixes and changes in it rather than rebuilding the system. Whereas if the system is redesigned then more quality will be maintained in it.
- Many compromises can be seen in the first version of the Prototype Model.
- Sometimes a software developer may make compromises in his implementation, just to get the prototype model up and running quickly, and after some time he may become comfortable with making such compromises and may forget that it is completely inappropriate to do so.

8. **Agile Model**

The meaning of Agile is swift or versatile. "Agile process model" refers to a software development approach based on iterative development. Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning. The project scope and requirements are laid down at the beginning of the development process. Plans regarding the number of iterations, the duration and the scope of each iteration are clearly defined in advance.

Each iteration is considered as a short time "frame" in the Agile process model, which typically lasts from one to four weeks. The division of the entire project into smaller parts helps to minimize the project risk and to reduce the overall project delivery time requirements. Each iteration involves a team working through a full software development life cycle including planning, requirements analysis, design, coding, and testing before a working product is demonstrated to the client.

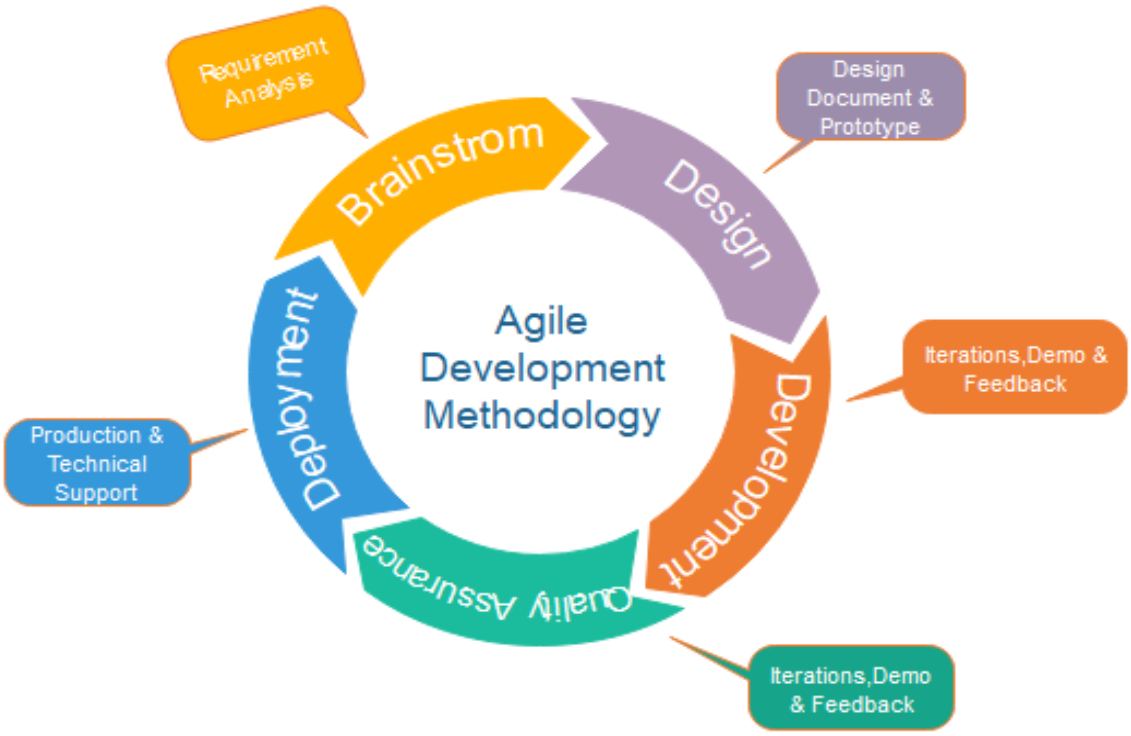


Fig. Agile Model

Phases of Agile Model:
Following are the phases in the Agile model are as follows:

1. Requirements gathering
2. Design the requirements
3. Construction/ iteration
4. Testing/ Quality assurance
5. Deployment
6. Feedback

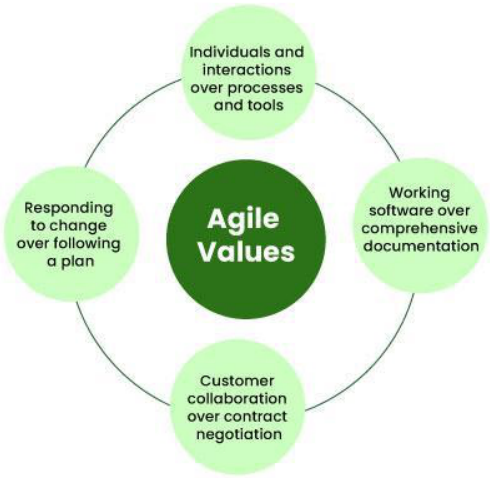
1. **Requirements gathering:** In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility.
2. **Design the requirements:** When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.
3. **Construction/ iteration:** When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.
4. **Testing:** In this phase, the Quality Assurance team examines the product's performance and looks for the bug.
5. **Deployment:** In this phase, the team issues a product for the user's work environment.
6. **Feedback:** After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

Agile software Development Model Methodologies/Frameworks:

- **Scrum:** This framework divides work into "sprints" (usually 1-4 weeks), where teams set goals, complete work, and review progress. Key roles are the Scrum Master, Product Owner, and Development Team.
- **Kanban:** Kanban visualizes work using a board with columns representing stages in the workflow (e.g., To-Do, In Progress, Done). It's particularly suitable for projects where tasks flow continuously.
- **Lean:** Derived from Lean Manufacturing, it focuses on reducing waste, maximizing value, and optimizing processes.
- **Extreme Programming (XP):** This emphasizes engineering practices like test-driven development, pair programming, and continuous integration.

4 Core Values of Agile Software Development

The Agile Software Development Methodology Manifesto describe four core values of Agile in software development.

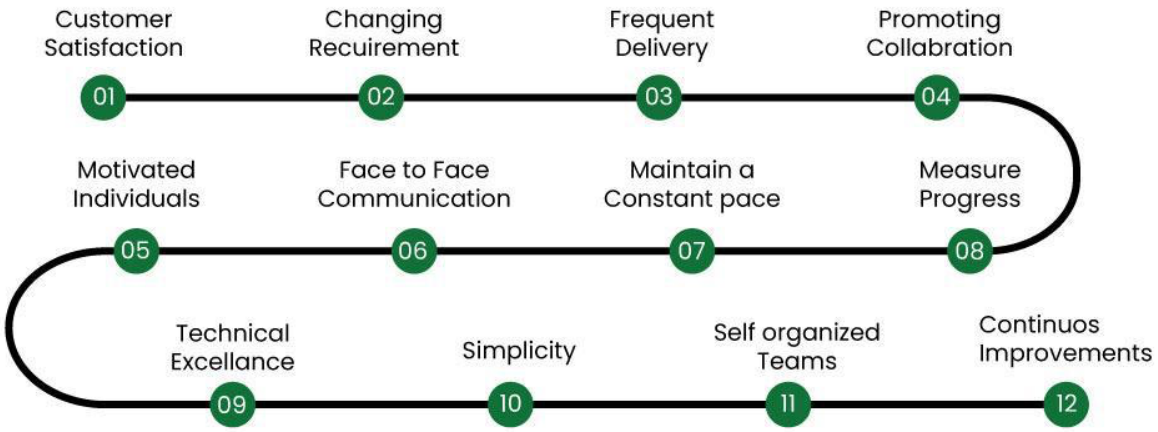


4 Values of Agile

1. Individuals and Interactions over Processes and Tools
2. Working Software over Comprehensive Documentation
3. Customer Collaboration over Contract Negotiation
4. Responding to Change over Following a Plan

12 Principles of Agile Software Development

The Agile Manifesto is based on four values and twelve principles that form the basis, for methodologies.

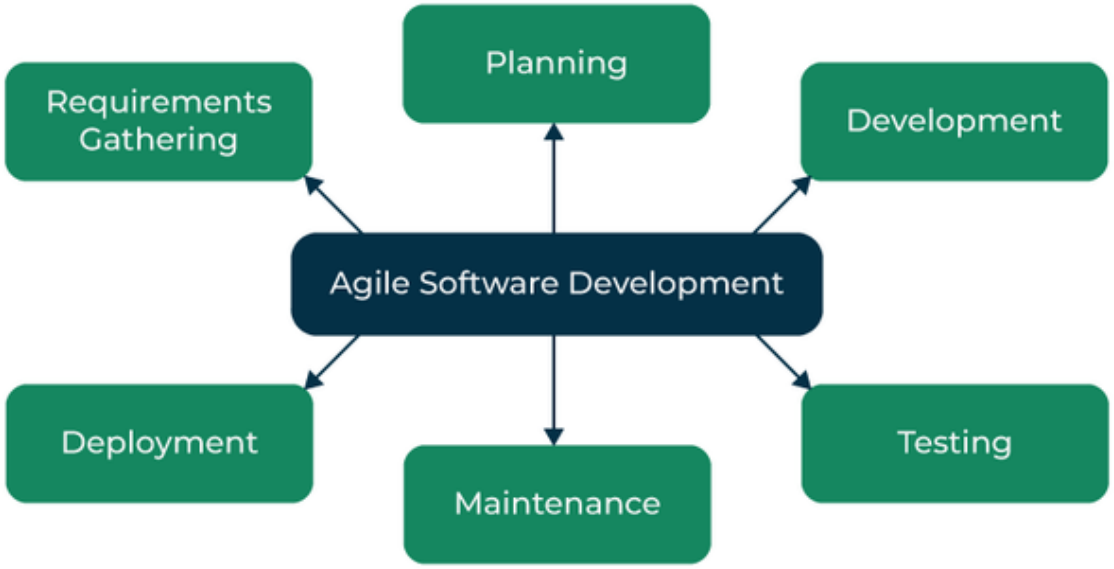


12 Principles of Agile Methodology

These principles include:

1. Ensuring customer satisfaction through the early delivery of software.
2. Being open to changing requirements in the stages of the development.
3. Frequently delivering working software with a main focus on preference for timeframes.
4. Promoting collaboration between business stakeholders and developers as an element.
5. Structuring the projects around individuals. Providing them with the necessary environment and support.
6. Prioritizing face to face communication whenever needed.
7. Considering working software as the measure of the progress.
8. Fostering development by allowing teams to maintain a pace indefinitely.
9. Placing attention on excellence and good design practices.
10. Recognizing the simplicity as crucial factor aiming to maximize productivity by minimizing the work.
11. Encouraging self-organizing teams as the approach to design and build systems.
12. Regularly reflecting on how to enhance effectiveness and to make adjustments accordingly.

The Agile Software Development Process



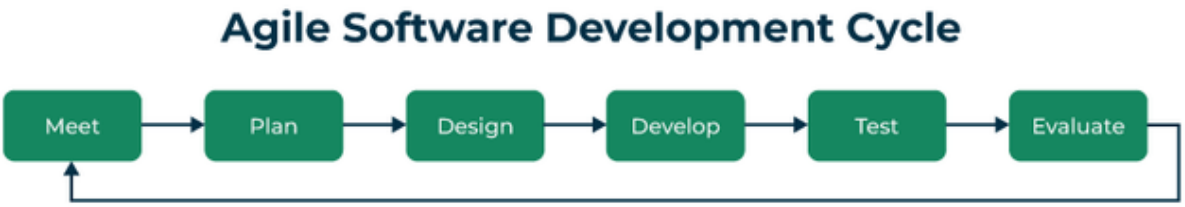
1. Requirement Gathering: The customer's requirements for the software are gathered and prioritized.
2. **Planning:** The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.
3. **Development:** The development team works to build the software, using frequent and rapid iterations.
4. **Testing:** The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.
5. **Deployment:** The software is deployed and put into use.
6. **Maintenance:** The software is maintained to ensure that it continues to meet the customer's needs and expectations.

Agile Software Development is widely used by software development teams and is considered to be a flexible and adaptable approach to software development that is well-suited to changing requirements and the fast pace of software development.
Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.

Agile Software development cycle

Let's see a brief overview of how development occurs in Agile philosophy.

1. concept
2. inception
3. iteration/construction
4. release
5. production
6. retirement



- **Step** 1: In the first step, concept, and business opportunities in each possible project are identified and the amount of time and work needed to complete the project is estimated. Based on their technical and financial viability, projects can then be prioritized and determined which ones are worthwhile pursuing.
- **Step** 2: In the second phase, known as inception, the customer is consulted regarding the initial requirements, team members are selected, and funding is secured. Additionally, a schedule outlining each team's responsibilities and the precise time at which each sprint's work is expected to be finished should be developed.
- **Step** 3: Teams begin building functional software in the third step, iteration/construction, based on requirements and ongoing feedback. Iterations, also known as single development cycles, are the foundation of the Agile software development cycle.

Advantages Agile Software Development

- Deployment of software is quicker and thus helps in increasing the trust of the customer.
- Can better adapt to rapidly changing requirements and respond faster.
- Helps in getting immediate feedback which can be used to improve the software in the next increment.
- People – Not Process. People and interactions are given a higher priority than processes and tools.
- Continuous attention to technical excellence and good design.
- **Increased collaboration and communication:** It emphasize collaboration and communication among team members, stakeholders, and customers. This leads to improved understanding, better alignment, and increased buy-in from everyone involved.
- **Flexibility and adaptability:** Agile methodologies are designed to be flexible and adaptable, making it easier to respond to changes in requirements, priorities, or market conditions. This allows teams to quickly adjust their approach and stay focused on delivering value.
- **Improved quality and reliability:** Agile methodologies place a strong emphasis on testing, quality assurance, and continuous improvement. This helps to ensure that software is delivered with high quality and reliability, reducing the risk of defects or issues that can impact the user experience.
- **Enhanced customer satisfaction:** Agile methodologies prioritize customer satisfaction and focus on delivering value to the customer. By involving customers throughout the development process, teams can ensure that the software meets their needs and expectations.
- **Increased team morale and motivation:** Agile methodologies promote a collaborative, supportive, and positive work environment. This can lead to increased team morale, motivation, and engagement, which can in turn lead to better productivity, higher quality work, and improved outcomes.

Disadvantages Agile Software Development

- In the case of large software projects, it is difficult to assess the effort required at the initial stages of the software development life cycle.
- Agile Development is more code-focused and produces less documentation.
- Agile development is heavily dependent on the inputs of the customer. If the customer has ambiguity in his vision of the outcome, it is highly likely that the project to get off track.
- Face-to-face communication is harder in large-scale organizations.
- Only senior programmers are capable of making the kind of decisions required during the development process. Hence, it's a difficult situation for new programmers to adapt to the environment.
- **Lack of predictability:** Agile Development relies heavily on customer feedback and continuous iteration, which can make it difficult to predict project outcomes, timelines, and budgets.
- Limited scope control: Agile Development is designed to be flexible and adaptable, which means that scope changes can be easily accommodated. However, this can also lead to scope creep and a lack of control over the project scope.
- **Lack of emphasis on testing:** Agile Development places a greater emphasis on delivering working code quickly, which can lead to a lack of focus on testing and quality assurance. This can result in bugs and other issues that may go undetected until later stages of the project.
- **Risk of team burnout:** Agile Development can be intense and fast-paced, with frequent sprints and deadlines. This can put a lot of pressure on team members and lead to burnout, especially if the team is not given adequate time for rest and recovery.
- **Lack of structure and governance:** Agile Development is often less formal and structured than other development methodologies, which can lead to a lack of governance and oversight. This can result in inconsistent processes and practices, which can impact project quality and outcomes.

Practices of Agile Software Development

- **Scrum:** Scrum is a framework for agile software development that involves iterative cycles called sprints, daily stand-up meetings, and a product backlog that is prioritized by the customer.
- **Kanban:** Kanban is a visual system that helps teams manage their work and improve their processes. It involves using a board with columns to represent different stages of the development process, and cards or sticky notes to represent work items.
- **Continuous Integration:** Continuous Integration is the practice of frequently merging code changes into a shared repository, which helps to identify and resolve conflicts early in the development process.
- **Test-Driven Development:** Test-Driven Development (TDD) is a development practice that involves writing automated tests before writing the code. This helps to ensure that the code meets the requirements and reduces the likelihood of defects.
- P**air Programming:** Pair programming involves two developers working together on the same code. This helps to improve code quality, share knowledge, and reduce the likelihood of defects.

Advantages of Agile Software Development over traditional software development approaches

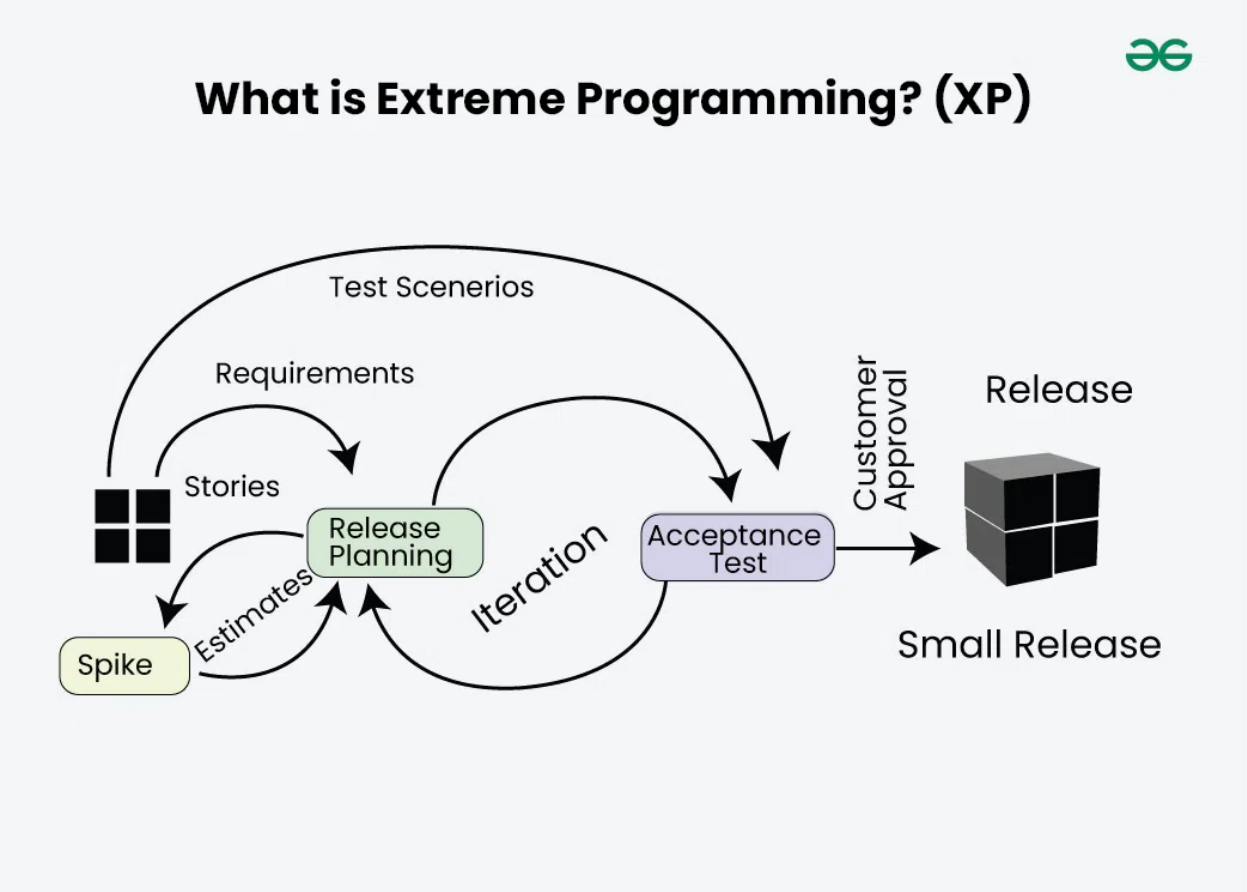
1. **Increased customer satisfaction:** Agile development involves close collaboration with the customer, which helps to ensure that the software meets their needs and expectations.
2. **Faster time-to-market:** Agile development emphasizes the delivery of working software in short iterations, which helps to get the software to market faster.
3. **Reduced risk:** Agile development involves frequent testing and feedback, which helps to identify and resolve issues early in the development process.
4. **Improved team collaboration:** Agile development emphasizes collaboration and communication between team members, which helps to improve productivity and morale.
5. **Adaptability to change:** Agile Development is designed to be flexible and adaptable, which means that changes to the project scope, requirements, and timeline can be accommodated easily. This can help the team to respond quickly to changing business needs and market demands.
6. **Better quality software:** Agile Development emphasizes continuous testing and feedback, which helps to identify and resolve issues early in the development process. This can lead to higher-quality software that is more reliable and less prone to errors.
7. **Increased transparency:** Agile Development involves frequent communication and collaboration between the team and the customer, which helps to improve transparency and visibility into the project status and progress. This can help to build trust and confidence with the customer and other stakeholders.
8. **Higher productivity:** Agile Development emphasizes teamwork and collaboration, which helps to improve productivity and reduce waste. This can lead to faster delivery of working software with fewer defects and rework.
9. **Improved project control:** Agile Development emphasizes continuous monitoring and measurement of project metrics, which helps to improve project control and decision-making. This can help the team to stay on track and make data-driven decisions throughout the development process.

When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

2.1. Extreme Programming [XP]:

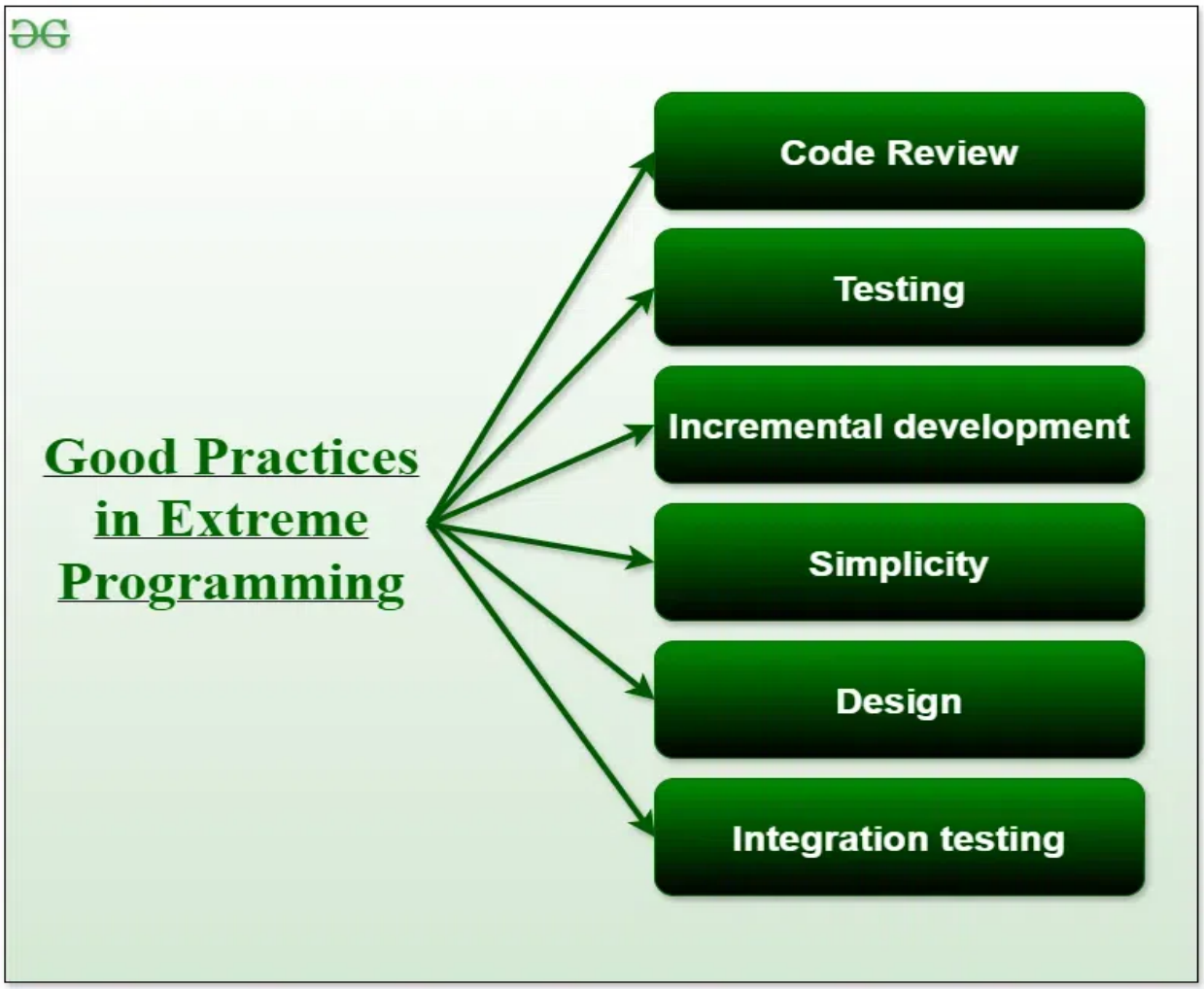
- Extreme Programming (XP) is an agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation.
- XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.



Extreme Programming (XP)

- Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy-based processes, particularly the waterfall approach.

Good Practices in Extreme Programming



Extreme Programming Good Practices

- **Code Review:** Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their work between them every hour.
- **Testing:** Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.
- **Incremental development:** Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.
- **Simplicity:** Simplicity makes it easier to develop good-quality code as well as to test and debug it.
- **Design:** Good quality design is important to develop good quality software. So, everybody should design daily.
- **Integration testing:** Integration Testing helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Basic Principles of Extreme programming

Some of the basic activities that are followed during software development by using the XP model are given below:

- **Coding:** The concept of coding which is used in the XP model is slightly different from traditional coding. Here, the coding activity includes drawing diagrams (modeling) that will be transformed into code, scripting a web-based system, and choosing among several alternative solutions.
- **Testing:** The XP model gives high importance to testing and considers it to be the primary factor in developing fault-free software.
- **Listening:** The developers need to carefully listen to the customers if they have to develop good quality software. Sometimes programmers may not have the depth knowledge of the system to be developed. So, the programmers should understand properly the functionality of the system and they have to listen to the customers.
- **Designing:** Without a proper design, a system implementation becomes too complex, and very difficult to understand the solution, thus making maintenance expensive. A good design results elimination of complex dependencies within a system. So, effective use of suitable design is emphasized.
- **Feedback:** One of the most important aspects of the XP model is to gain feedback to understand the exact customer needs. Frequent contact with the customer makes the development effective.
- **Simplicity:** The main principle of the XP model is to develop a simple system that will work efficiently in the present time, rather than trying to build something that would take time and may never be used. It focuses on some specific features that are immediately needed, rather than engaging time and effort on speculations of future requirements.
- **Pair Programming:** XP encourages pair programming where two developers work together at the same workstation. This approach helps in knowledge sharing, reduces errors, and improves code quality.
- **Continuous Integration:** In XP, developers integrate their code into a shared repository several times a day. This helps to detect and resolve integration issues early on in the development process.
- **Refactoring:** XP encourages refactoring which is the process of restructuring existing code to make it more efficient and maintainable. Refactoring helps to keep the codebase clean, organized, and easy to understand.
- **Collective Code Ownership:** In XP, there is no individual ownership of code. Instead, the entire team is responsible for the codebase. This approach ensures that all team members have a sense of ownership and responsibility towards the code.
- **Planning Game:** XP follows a planning game, where the customer and the development team collaborate to prioritize and plan development tasks. This approach helps to ensure that the team is working on the most important features and delivers value to the customer.
- **On-site Customer:** XP requires an on-site customer who works closely with the development team throughout the project. This approach helps to ensure that the customer's needs are understood and met, and also facilitates communication and feedback.

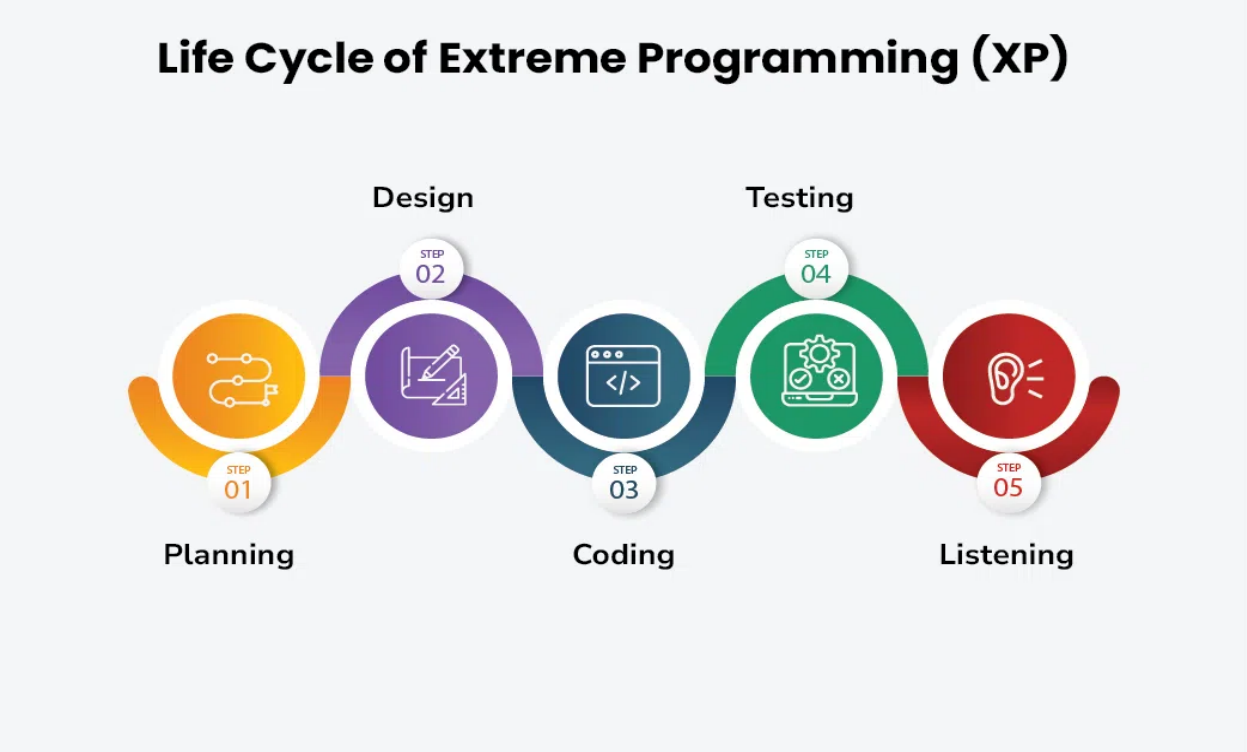
Applications of Extreme Programming (XP)

Some of the projects that are suitable to develop using the XP model are given below:

- **Small projects:** The XP model is very useful in small projects consisting of small teams as face-to-face meeting is easier to achieve.
- **Projects involving new technology or Research projects:** This type of project faces changing requirements rapidly and technical problems. So, XP model is used to complete this type of project.
- **Web development projects:** The XP model is well-suited for web development projects as the development process is iterative and requires frequent testing to ensure the system meets the requirements.
- **Collaborative projects:** The XP model is useful for collaborative projects that require close collaboration between the development team and the customer.
- **Projects with tight deadlines:** The XP model can be used in projects that have a tight deadline, as it emphasizes simplicity and iterative development.
- **Projects with rapidly changing requirements: The** XP model is designed to handle rapidly changing requirements, making it suitable for projects where requirements may change frequently.
- **Projects where quality is a high priority:** The XP model places a strong emphasis on testing and quality assurance, making it a suitable approach for projects where quality is a high priority.

Life Cycle of Extreme Programming (XP)

The Extreme Programming Life Cycle consist of five phases:

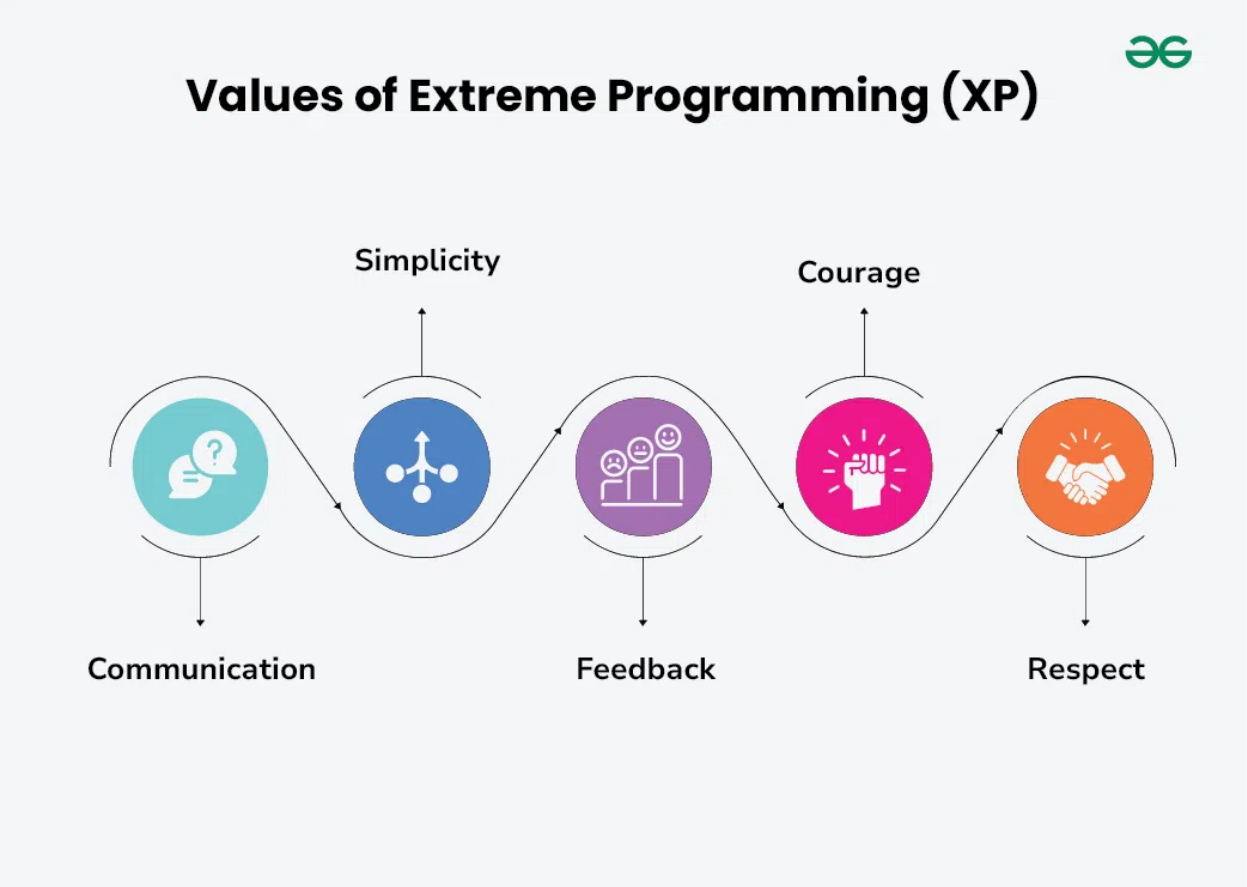


Life Cycle of Extreme Programming (XP)

1. **Planning:** The first stage of Extreme Programming is planning. During this phase, clients define their needs in concise descriptions known as user stories. The team calculates the effort required for each story and schedules releases according to priority and effort.
2. **Design:** The team creates only the essential design needed for current user stories, using a common analogy or story to help everyone understand the overall system architecture and keep the design straightforward and clear.
3. **Coding:** Extreme Programming (XP) promotes pair programming i.e**.** wo developers work together at one workstation, enhancing code quality and knowledge sharing. They write tests before coding to ensure functionality from the start (TDD), and frequently integrate their code into a shared repository with automated tests to catch issues early.
4. **Testing:** Extreme Programming (XP) gives more importance to testing that consist of both unit tests and acceptance test**.** Unit tests, which are automated, check if specific features work correctly. Acceptance tests, conducted by customers, ensure that the overall system meets initial requirements. This continuous testing ensures the software's quality and alignment with customer needs.
5. **Listening:** In the listening phase regular feedback from customers to ensure the product meets their needs and to adapt to any changes.

Values of Extreme Programming (XP)

There are five core values of Extreme Programming (XP)



Values of Extreme Programming (XP)

1. **Communication:** The essence of communication is for information and ideas to be exchanged amongst development team members so that everyone has an understanding of the system requirements and goals. Extreme Programming (XP) supports this by allowing open and frequent communication between members of a team.
2. **Simplicity:** Keeping things as simple as possible helps reduce complexity and makes it easier to understand and maintain the code.
3. **Feedback:** Feedback loops which are constant are among testing as well as customer involvements which helps in detecting problems earlier during development.
4. **Courage:** Team members are encouraged to take risks, speak up about problems, and adapt to change without fear of repercussions.
5. **Respect**: Every member's input or opinion is appreciated which promotes a collective way of working among people who are supportive within a certain group.

Advantages of Extreme Programming (XP)

- **Slipped schedules:** Timely delivery is ensured through slipping timetables and doable development cycles.
- **Misunderstanding the business and/or domain** − Constant contact and explanations are ensured by including the client on the team.
- **Cancelled projects:** Focusing on ongoing customer engagement guarantees open communication with the consumer and prompt problem-solving.
- **Staff turnover:** Teamwork that is focused on cooperation provides excitement and goodwill. Team spirit is fostered by multidisciplinary cohesion.
- **Costs incurred in changes:** Extensive and continuing testing ensures that the modifications do not impair the functioning of the system. A functioning system always guarantees that there is enough time to accommodate changes without impairing ongoing operations.
- **Business changes:** Changes are accepted at any moment since they are seen to be inevitable.
- **Production and post-delivery defects:** the unit tests to find and repair bugs as soon as possible.

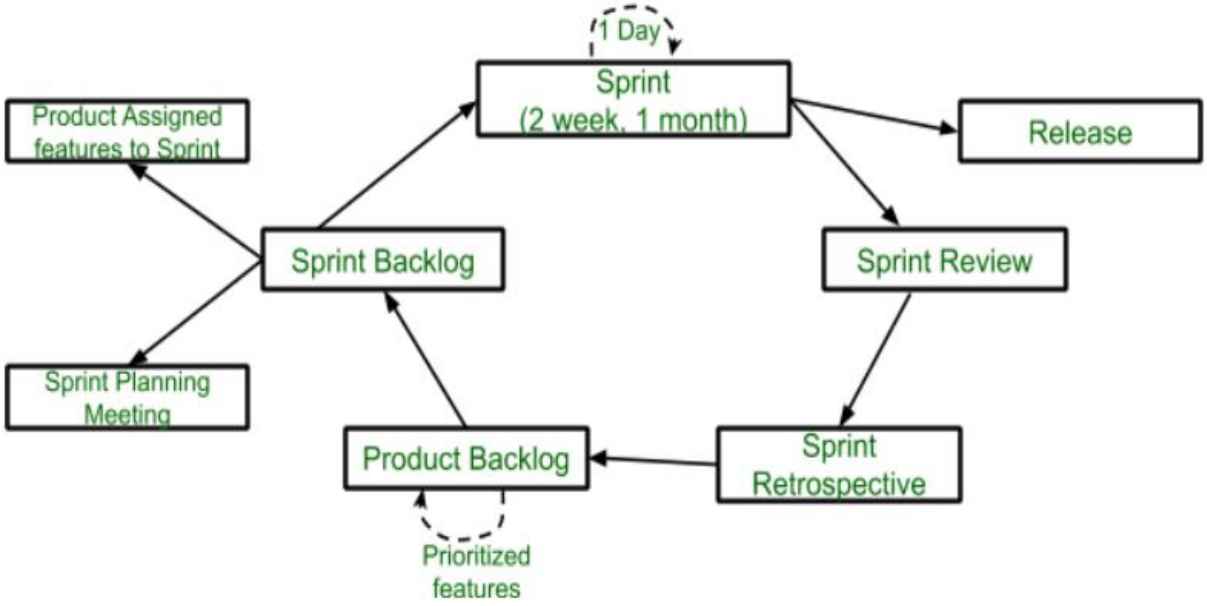
2.2. Scrum:

- Scrum is a management framework that teams use to self-organize tasks and work towards a common goal.
- It is a framework within which people can address complex adaptive problems while the productivity and creativity of delivering products are at the highest possible value.
- Scrum allows us to develop products of the highest value while making sure that we maintain creativity and productivity.
- The iterative and incremental approach used in scrum allows the teams to adapt to the changing requirements.

Silent features of Scrum

- Scrum is a light-weighted framework
- Scrum emphasizes self-organization
- Scrum is simple to understand
- Scrum framework helps the team to work together

Lifecycle of Scrum



- **Sprint:** A Sprint is a time box of one month or less. A new Sprint starts immediately after the completion of the previous Sprint. **Release:** When the product is completed, it goes to the Release stage.
- **Sprint Review:** If the product still has some non-achievable features, it will be checked in this stage and then passed to the Sprint Retrospective stage.
- **Sprint Retrospective**: In this stage quality or status of the product is checked.
- **Product Backlog:** According to the prioritize features the product is organized.
- **Sprint Backlog:** Sprint Backlog is divided into two parts Product assigned features to sprint and Sprint planning meeting.

Advantage of Scrum framework

- Scrum framework is fast moving and money efficient.
- Scrum framework works by dividing the large product into small sub-products. It's like a divide and conquer strategy
- In Scrum customer satisfaction is very important.
- Scrum is adaptive in nature because it have short sprint.
- As Scrum framework rely on constant feedback therefore the quality of product increases in less amount of time

Disadvantage of Scrum framework

- Scrum framework do not allow changes into their sprint.
- Scrum framework is not fully described model. If you want to adopt it you need to fill in the framework with your own details like XP, DSDM, Kanban.
- It can be difficult for the Scrum to plan, structure and organize a project that lacks a clear definition.
- The daily Scrum meetings and frequent reviews require substantial resources.

2.3. Agile Project Management:

1. Agile Project Management:

- Agile project management is an iterative and flexible approach to managing projects, initially popularized in software development but now used across various industries.
- Agile emphasizes delivering value quickly, adapting to changes, and continuously improving through collaboration among cross-functional teams.
- Instead of rigid plans, Agile relies on iterative cycles (often called "sprints") and incremental deliveries, allowing teams to adapt to changing requirements and feedback rapidly.

Core Principles of Agile Project Management

The Agile approach is built on principles outlined in the Agile Manifesto, which emphasizes:

1. **Individuals and interactions** over processes and tools.
2. **Working software** (or product) over comprehensive documentation.
3. **Customer collaboration** over contract negotiation.
4. **Responding to change** over following a fixed plan.

Benefits of Agile Project Management:

- **Faster time to market:** Agile enables rapid delivery of value.
- **Improved customer satisfaction:** Agile fosters collaboration and responsiveness to customer needs.
- **Increased flexibility:** Agile allows for adapting to changing requirements.
- **Higher quality:** Agile emphasizes continuous improvement and testing.
- **Enhanced team morale:** Agile empowers teams and promotes ownership.

Challenges of Agile:

- **Cultural shift:** Adopting Agile requires a change in mindset and organizational culture.
- **Skill development:** Teams need training and coaching to effectively implement Agile practices.
- **Measuring success:** Traditional metrics may not be suitable for Agile projects.
- **Managing dependencies:** In large-scale projects, coordinating dependencies can be complex.

2. Scaling Agile Methods:

- Scaling Agile becomes crucial when applying Agile practices across large organizations or projects involving multiple teams.
- Traditional Agile frameworks work best with small teams, but large enterprises need coordination and alignment. Some popular scaling frameworks include:

1. **SAFe (Scaled Agile Framework):** SAFe provides a structured approach to scaling Agile across large organizations, with multiple layers like Team, Program, and Portfolio. It introduces Program Increments (PIs) to align multiple teams.
2. **LeSS (Large Scale Scrum):** LeSS scales Scrum for multiple teams by using one Product Backlog across multiple teams, each working in parallel while maintaining a single Scrum Master and a shared Product Owner role.
3. **Spotify Model:** Developed by Spotify, this model organizes people into "squads" (similar to Scrum teams) and groups them into "tribes" with aligned goals. This model emphasizes autonomy and alignment, allowing teams to self-organize.
4. **Disciplined Agile Delivery (DAD):** DAD provides a decision-making framework that helps teams choose between various Agile and Lean techniques based on the project needs. It emphasizes a holistic view, considering areas like architecture and governance.

2.4. Pros and Cons of Process Models and their Applicability:

- Process models are essential tools in project management, offering structured approaches to software development and system design.
- However, their effectiveness depends on their appropriate application and understanding of their inherent strengths and weaknesses.

Common Process Models

- **Waterfall Model:** A linear, sequential approach, well-suited for projects with well-defined requirements.
- **Agile Model:** An iterative and incremental approach, ideal for projects with evolving requirements.
- **Spiral Model:** A risk-driven approach, balancing iterative development with systematic risk management.
- **V-Model:** A software development lifecycle model that emphasizes verification and validation at each stage.

Pros and Cons of Process Models

Pros:

- **Structure and Organization:** Models provide a clear framework for project planning, execution, and monitoring.
- **Risk Management:** They help identify and mitigate potential risks early in the project lifecycle.
- **Quality Assurance:** Models promote adherence to quality standards and best practices.
- **Team Collaboration:** They facilitate effective communication and coordination among team members.
- **Customer Satisfaction:** By following a structured approach, project teams can deliver products that meet customer expectations.

Cons:

- **Rigidity:** Some models, like the Waterfall model, can be inflexible and resistant to change.
- **Overemphasis on Documentation:** Excessive documentation can be time-consuming and may not always add value.
- **Limited Adaptability:** Traditional models may struggle to accommodate evolving requirements and unforeseen challenges.
- **Complexity:** Some models can be overly complex and difficult to understand and implement.

Applicability of Process Models:

The choice of a process model depends on various factors, including:

- **Project Size and Complexity:** Larger, more complex projects may benefit from a structured approach like Waterfall or V-Model.
- **Requirement Certainty:** Projects with well-defined requirements are better suited for linear models like Waterfall.
- **Customer Involvement:** Agile models are ideal for projects with high customer involvement and frequent feedback.
- **Risk Tolerance:** Risk-averse projects may benefit from a risk-driven approach like the Spiral model.
- **Team Experience and Maturity:** Experienced teams may be able to adapt to more flexible models like Agile.

Hybrid Approach

- In practice, a hybrid approach that combines elements of different models can often be the most effective.
- This allows teams to tailor their processes to specific project needs, balancing structure and flexibility.

2.5. Component-Based Software Engineering (CBSE)

- Component-Based Software Engineering (CBSE) is a process that focuses on the design and development of computer-based systems with the use of reusable software components.
- It not only identifies candidate components but also qualifies each component's interface, adapts components to remove architectural mismatches, assembles components into a selected architectural style, and updates components as requirements for the system change.

Component-based software engineering (CBSE) is an approach to software development emerged in the 1990's that relies on the reuse of entities called 'software components'. It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific. Components are more abstract than object classes and can be considered to be stand-alone service providers. They can exist as stand-alone entities.

CBSE essentials

- **Independent components** specified by their interfaces.
- **Component standards** to facilitate component integration.
- **Middleware** that provides support for component inter-operability.
- **A development process** that is geared to reuse.

Apart from the benefits of reuse, CBSE is based on sound **software engineering design principles**:

- Components are independent so do not interfere with each other;
- Component implementations are hidden;
- Communication is through well-defined interfaces;
- One component can be replaced by another if its interface is maintained;
- Component infrastructures offer a range of standard services.

Standards need to be established so that components can communicate with each other and inter-operate. Unfortunately, several competing component standards were established: Sun's Enterprise Java Beans, Microsoft's COM and .NET, CORBA's CCM. In practice, these multiple standards have hindered the uptake of CBSE. It is impossible for components developed using different approaches to work together.

Solution for interoperating standards: **component as a service**. An executable service is a type of independent component. It has a 'provides' interface but not a 'requires' interface. From the outset, services have been based around standards so there are no problems in communicating between services offered by different vendors. System performance may be slower with services but this approach is replacing CBSE in many systems.

Components and component models:

Components provide a service without regard to where the component is executing or its programming language. A component is an **independent executable entity** that can be made up of one or more executable objects. The component interface is published and all interactions are through the published interface.

Component characteristics:

Composable

For a component to be composable, all external interactions must take place through publicly defined interfaces. In addition, it must provide external access to information about itself, such as its methods and attributes.

Deployable

To be deployable, a component has to be self-contained. It must be able to operate as a stand-alone entity on a component platform that provides an implementation of the component model. This usually means that the component is binary and does not have to be compiled before it is deployed. If a component is implemented as a service, it does not have to be deployed by a user of that component. Rather, it is deployed by the service provider.

Documented

Components have to be fully documented so that potential users can decide whether or not the components meet their needs. The syntax and, ideally, the semantics of all component interfaces should be specified.

Independent

A component should be independent--it should be possible to compose and deploy it without having to use other specific components. In situations where the component needs externally provided services, these should be explicitly set out in a "requires" interface specification.

Standardized

Component standardization means that a component used in a CBSE process has to conform to a standard component model. This model may define component interfaces, component metadata, documentation, composition, and deployment.

The component is an independent, executable entity. It does not have to be compiled before it is used with other components. The services offered by a component are made available through an interface and all component interactions take place through that interface. The component interface is expressed in terms of parameterized operations and its internal state is never exposed.

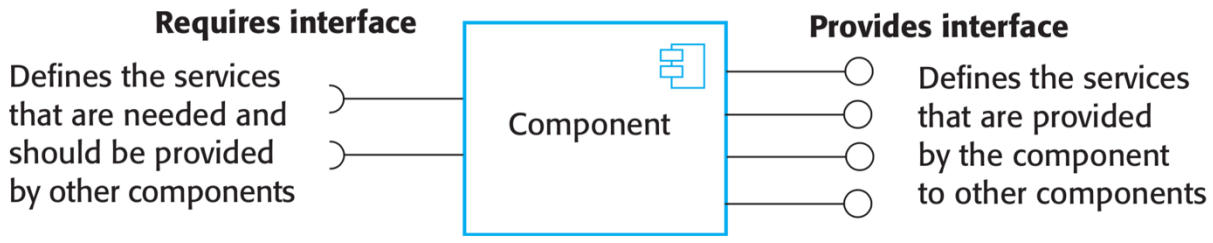
Component interfaces:

"Provides" interface

Defines the services that are provided by the component to other components. This interface, essentially, is the component API. It defines the methods that can be called by a user of the component.

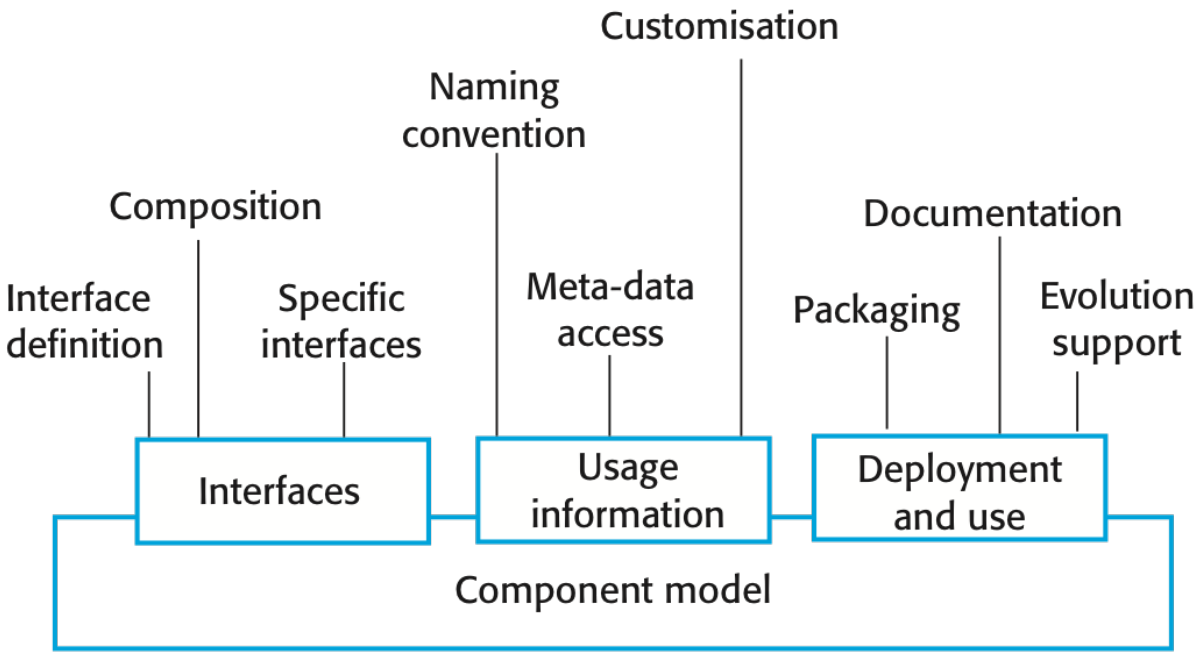
"Requires" interface

Defines the services that specifies what services must be made available for the component to execute as specified. This does not compromise the independence or deployability of a component because the 'requires' interface does not define how these services should be provided.



Components are accessed using remote procedure calls (RPCs). Each component has a unique identifier (usually a URL) and can be referenced from any networked computer. Therefore it can be called in a similar way as a procedure or method running on a local computer.

A **component model** is a definition of standards for component implementation, documentation and deployment. Examples of component models: EJB model (Enterprise Java Beans), COM+ model (.NET model), Corba Component Model. The component model specifies how interfaces should be defined and the elements that should be included in an interface definition.



Elements of a component model:

Interfaces

Components are defined by specifying their interfaces. The component model specifies how the interfaces should be defined and the elements, such as operation names, parameters and exceptions, which should be included in the interface definition.

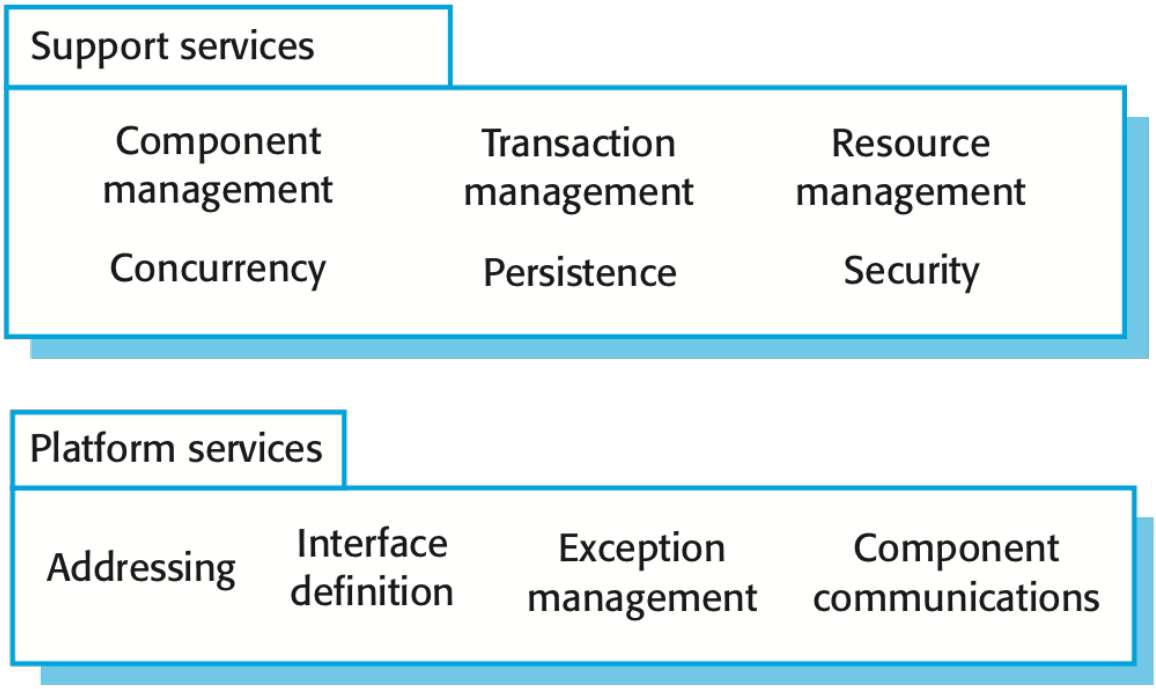
Usage

In order for components to be distributed and accessed remotely, they need to have a unique name or handle associated with them. This has to be globally unique.

Deployment

The component model includes a specification of how components should be packaged for deployment as independent, executable entities.

Component models are the basis for middleware that provides support for executing components. Component model implementations provide: **platform services** that allow components written according to the model to communicate, and **support services** that are application-independent services used by different components. To use services provided by a model, components are deployed in a **container**. This is a set of interfaces used to access the service implementations.



CBSE processes

CBSE processes are software processes that support component-based software engineering. They take into account the possibilities of reuse and the different process activities involved in developing and using reusable components. There are two types of CBSE processes:

- **CBSE for reuse** is concerned with developing components or services that will be reused in other applications. It usually involves generalizing existing components.
- **CBSE with reuse** is the process of developing new applications using existing components and services.

CBSE for reuse focuses on component and service development. Components developed for a specific application usually have to be generalised to make them reusable. A component is most likely to be reusable if it associated with a stable domain abstraction (business object). For example, in a hospital stable domain abstractions are associated with the fundamental purpose - nurses, patients, treatments, etc.

Component reusability:

- Should reflect stable domain abstractions;
- Should hide state representation;
- Should be as independent as possible;
- Should publish exceptions through the component interface.

There is a **trade-off between reusability and usability**. The more general the interface, the greater the reusability but it is then more complex and hence less usable.

To make an existing component reusable:

- Remove application-specific methods.
- Change names to make them general.
- Add methods to broaden coverage.
- Make exception handling consistent.
- Add a configuration interface for component adaptation.
- Integrate required components to reduce dependencies.

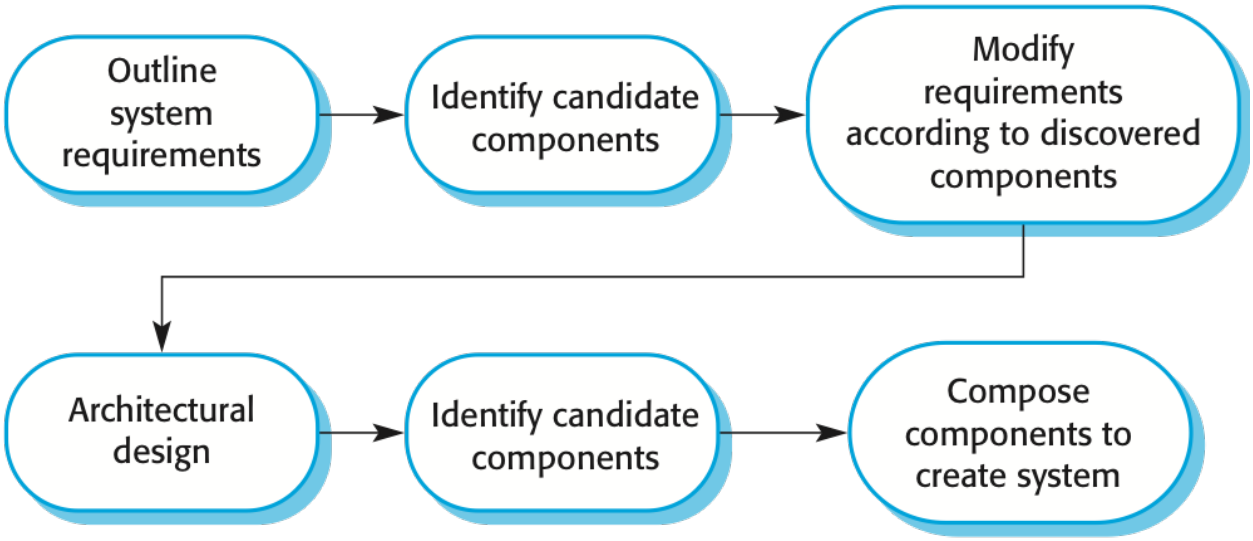
Existing **legacy systems** that fulfill a useful business function can be re-packaged as components for reuse. This involves writing a wrapper component that implements provides and requires interfaces then accesses the legacy system. Although costly, this can be much less expensive than rewriting the legacy system.

The **development cost** of reusable components may be higher than the cost of specific equivalents. This extra reusability enhancement cost should be an organization rather than a project cost. Generic components may be less space-efficient and may have longer execution times than their specific equivalents.

Component management involves deciding how to classify the component so that it can be discovered, making the component available either in a repository or as a service, maintaining information about the use of the component and keeping track of different component versions. A company with a reuse program may carry out some form of component certification before the component is made available for reuse. Certification means that someone apart from the developer checks the quality of the component.

CBSE with reuse process has to find and integrate reusable components. When reusing components, it is essential to make trade-offs between ideal requirements and the services actually provided by available components. This involves:

- Developing outline requirements;
- Searching for components then modifying requirements according to available functionality;
- Searching again to find if there are better components that meet the revised requirements;
- Composing components to create the system.



Component identification issues:

- You need to be able to **trust** the supplier of a component. At best, an untrusted component may not operate as advertised; at worst, it can breach your security.
- Different groups of components will satisfy different **requirements**.
- **Validation.** The component specification may not be detailed enough to allow comprehensive tests to be developed. Components may have unwanted functionality. How can you test this will not interfere with your application?

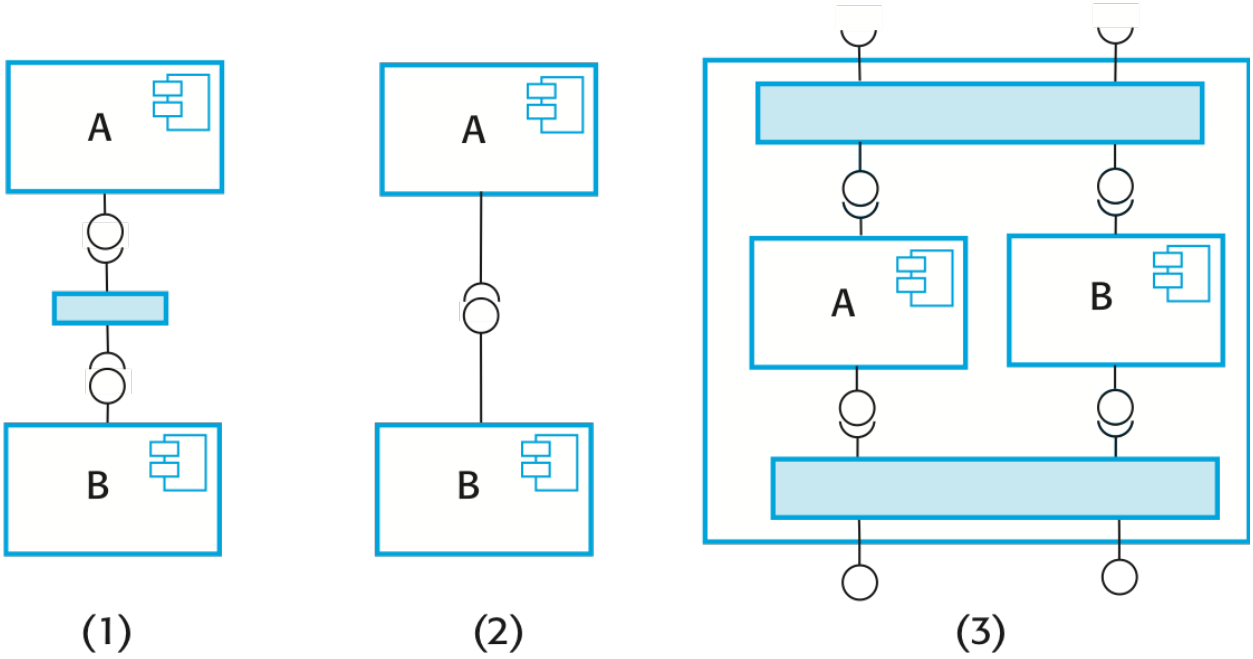
Component validation involves developing a set of test cases for a component (or, possibly, extending test cases supplied with that component) and developing a test harness to run component tests. The major problem with component validation is that the component specification may not be sufficiently detailed to allow you to develop a complete set of component tests. As well as testing that a component for reuse does what you require, you may also have to check that the component does not include any malicious code or functionality that you don't need.

Component composition

Component composition is the process of **assembling components to create a system**. Composition involves integrating components with each other and with the component infrastructure. Normally you have to write 'glue code' to integrate components.

Three types of component composition:

1. **Sequential composition** where the composed components are executed in sequence. This involves composing the provides interfaces of each component.
2. **Hierarchical composition** where one component calls on the services of another. The provides interface of one component is composed with the requires interface of another.
3. **Additive composition** where the interfaces of two components are put together to create a new component. Provides and requires interfaces of integrated component is a combination of interfaces of constituent components.



When components are developed independently for reuse, their interfaces are often incompatible. Three types of incompatibility can occur:

- **Parameter incompatibility** where operations have the same name but are of different types.
- **Operation incompatibility** where the names of operations in the composed interfaces are different.
- **Operation incompleteness** where the provides interface of one component is a subset of the requires interface of another.

Adaptor components address the problem of component incompatibility by reconciling the interfaces of the components that are composed. Different types of adaptor are required depending on the type of composition.

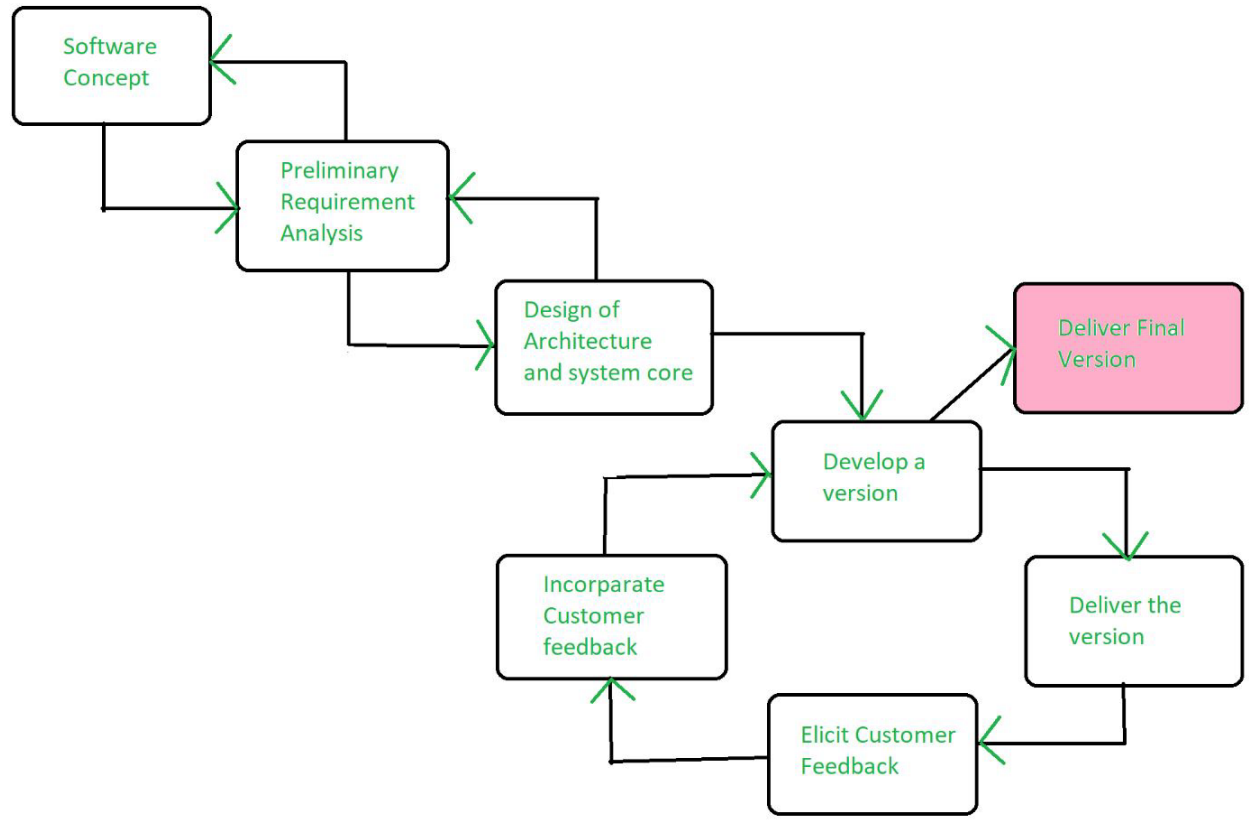
When composing components, you may find conflicts between functional and non-functional requirements, and conflicts between the need for rapid delivery and system evolution. You need to make decisions such as:

- What composition of components is effective for delivering the functional requirements?
- What composition of components allows for future change?
- What will be the emergent properties of the composed system?

2.6. Evolutionary Model:

- The evolutionary model is a combination of the Iterative and Incremental models of the software development life cycle.
- It is better for software products that have their feature sets redefined during development because of user feedback and other factors.
- The Evolutionary development model divides the development cycle into smaller, incremental waterfall models in which users can get access to the product at the end of each cycle.
- Feedback is provided by the users on the product for the planning stage of the next cycle and the development team responds, often by changing the product, plan, or process.

The model allows for changing requirements as well as all work is broken down into maintainable work chunks.



Application of Evolutionary Model

- It is used in large projects where you can easily find modules for incremental implementation. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
- Evolutionary model is also used in object-oriented software development because the system can be easily portioned into units in terms of objects.

Necessary Conditions for Implementing this Model

- Customer needs are clear and been explained in deep to the developer team.
- There might be small changes required in separate parts but not a major change.
- As it requires time, so there must be some time left for the market constraints.
- Risk is high and continuous targets to achieve and report to customer repeatedly.
- It is used when working on a technology is new and requires time to learn.

Advantages Evolutionary Model

- **Adaptability to Changing Requirements:** Evolutionary models work effectively in projects when the requirements are ambiguous or change often. They support adjustments and flexibility along the course of development.
- **Early and Gradual Distribution:** Functional components or prototypes can be delivered early thanks to incremental development. Faster user satisfaction and feedback may result from this.
- **User Commentary and Involvement:** Evolutionary models place a strong emphasis on ongoing user input and participation. This guarantees that the software offered closely matches the needs and expectations of the user.
- **Improved Handling of Difficult Projects:** Big, complex tasks can be effectively managed with the help of evolutionary models. The development process is made simpler by segmenting the project into smaller, easier-to-manage portions.

Disadvantages Evolutionary Model

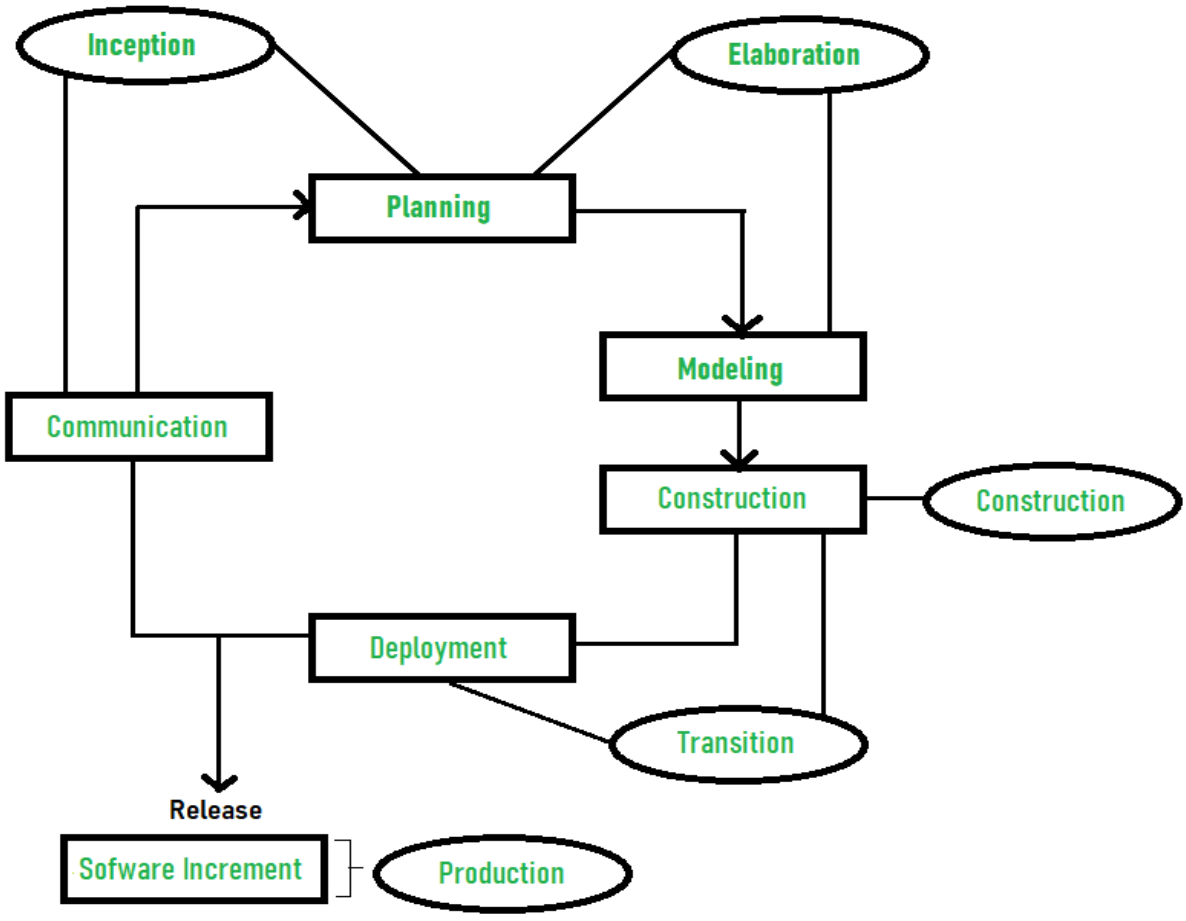
- **Communication Difficulties:** Evolutionary models require constant cooperation and communication. The strategy may be less effective if there are gaps in communication or if team members are spread out geographically.
- **Dependence on an Expert Group:** A knowledgeable and experienced group that can quickly adjust to changes is needed for evolutionary models. Teams lacking experience may find it difficult to handle these model's dynamic nature.
- **Increasing Management Complexity:** Complexity can be introduced by organizing and managing several increments or iterations, particularly in large projects. In order to guarantee integration and synchronization, good project management is needed.
- **Greater Initial Expenditure:** As evolutionary models necessitate continual testing, user feedback and prototyping, they may come with a greater starting cost. This may be a problem for projects that have limited funding.

2.7. **Rational Unified Process (RUP):**

- Rational Unified Process (RUP) is a software development process for object-oriented models. It is also known as the Unified Process Model.
- It is created by Rational Corporation and is designed and documented using UML (Unified Modeling Language).
- This process is included in the IBM Rational Method Composer (RMC) product. IBM (International Business Machine Corporation) allows us to customize, design, and personalize the unified process.
- RUP is proposed by Ivar Jacobson, Grady Bootch, and James Rambaugh.
- Some characteristics of RUP include being use-case driven, Iterative (repetition of the process), incremental (increase in value) by nature, delivered online using web technology, can be customized or tailored in modular and electronic form, etc. RUP reduces unexpected development costs and prevents the wastage of resources.

Phases of RUP

There is a total of five phases of the life cycle of RUP:
Inception
Elaboration
Construction
Transition
Production



Inception -
Communication and planning are the main ones.
Identifies the scope of the project using a use-case model allowing managers to estimate costs and time required.
Customers' requirements are identified and then it becomes easy to make a plan for the project.
The project plan, Project goal, risks, use-case model, and Project description, are made.
The project is checked against the milestone criteria and if it couldn't pass these criteria then the project can be either cancelled or redesigned.
Elaboration -
Planning and modeling are the main ones.
A detailed evaluation and development plan is carried out and diminishes the risks.
Revise or redefine the use-case model (approx. 80%), business case, and risks.
Again, checked against milestone criteria and if it couldn't pass these criteria then again project can be cancelled or redesigned.
Executable architecture baseline.
Construction -
The project is developed and completed.
System or source code is created and then testing is done.
Coding takes place.
Transition -
The final project is released to the public.
Transit the project from development into production.
Update project documentation.
Beta testing is conducted.
Defects are removed from the project based on feedback from the public.
Production -
The final phase of the model.
The project is maintained and updated accordingly.

Advantages of Rational Unified Process (RUP)
Following are the advantages of Rational Unified Process (RUP):

- RUP provides good documentation; it completes the process in itself.
- RUP provides risk-management support.
- RUP reuses the components, and hence total time duration is less.
- Good online support is available in the form of tutorials and training.

Disadvantages of Rational Unified Process (RUP)
Following are the disadvantages of Rational Unified Process (RUP):

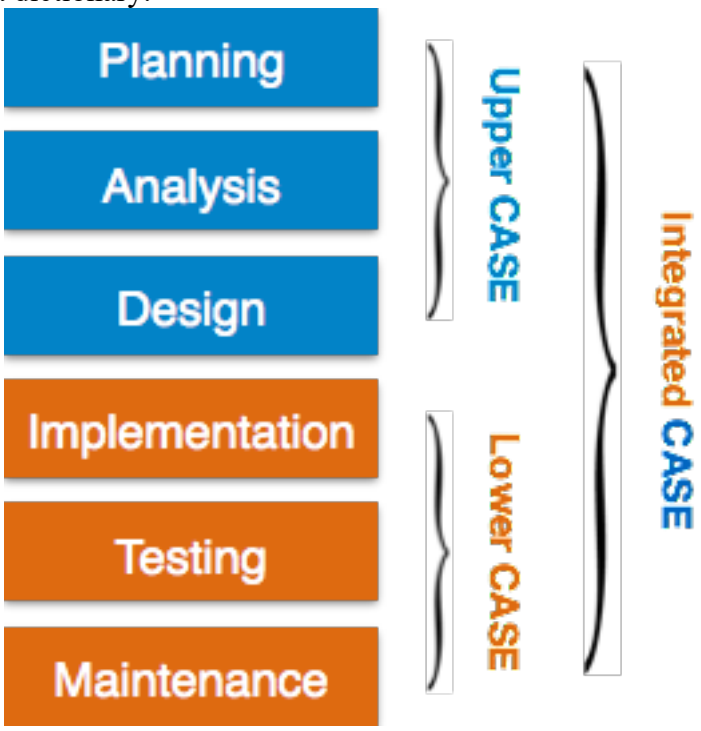
- Team of expert professional is required, as the process is complex.
- Complex and not properly organized process.
- More dependency on risk management.
- Hard to integrate again and again.
- Rational Unified Process (RUP) Best Practices

2.8. OVERVIEW AND CLASSIFICATION OF CASE TOOLS:

- CASE (Computer-Aided Software Engineering) tools are software programs designed to help with various tasks in software development.
- They automate processes across the Software Development Life Cycle, improving productivity, quality, and project management.

Components of CASE Tools

- **Central Repository** - CASE tools require a central repository, which can serve as a source of common, integrated and consistent information. Central repository is a central place of storage where product specifications, requirement documents, related reports and diagrams, other useful information regarding management is stored. Central repository also serves as data dictionary.



- **Upper Case Tools** - Upper CASE tools are used in planning, analysis and design stages of SDLC.
- **Lower Case Tools** - Lower CASE tools are used in implementation, testing and maintenance.
- **Integrated Case Tools** - Integrated CASE tools are helpful in all the stages of SDLC, from Requirement gathering to Testing and documentation.

Case Tools Types

1. **Diagram tools:**
   - These tools are used to represent system components, data and control flow among various software components and system structure in a graphical form. For example, Flow Chart Maker tool for creating state-of-the-art flowcharts.

2. **Process Modeling Tools:**
   - Process modeling is method to create software process model, which is used to develop the software. For example, EPF Composer

3. **Project Management Tools:**
   - These tools are used for project planning, cost and effort estimation, project scheduling and resource planning. For example, Creative Pro Office, Trac Project, Basecamp.

4. **Documentation Tools:**
   - Documentation in a software project starts prior to the software process, goes throughout all phases of SDLC and after the completion of the project.
   - For example, Doxygen, DrExplain, Adobe RoboHelp for documentation.

5. **Analysis Tools:**
   - These tools help to gather requirements, automatically check for any inconsistency, inaccuracy in the diagrams, data redundancies or erroneous omissions. For example, Accept 360, Accompa, CaseComplete for requirement analysis, Visible Analyst for total analysis.

6. **Design Tools:**
   - These tools help software designers to design the block structure of the software, which may further be broken down in smaller modules using refinement techniques. For example, Animated Software Design

7. **Configuration Management Tools:**
   - Configuration Management tools deal with Version and revision management Baseline configuration management Change control management
   - CASE tools help in this by automatic tracking, version management and release management. For example, Fossil, Git, Accu REV.

8. **Change Control Tools**
   - These tools are considered as a part of configuration management tools. They deal with changes made to the software after its baseline is fixed or when the software is first released.

9. **Programming Tools:**
   - These tools consist of programming environments like IDE (Integrated Development Environment), in-built modules library and simulation tools. For example, Cscope to search code in C, Eclipse.

10. Prototyping Tools:

- Software prototype is simulated version of the intended software product. Prototype provides initial look and feel of the product and simulates few aspect of actual product.
- For example, Serena prototype composer, Mockup Builder.

11. Web Development Tools:

- These tools assist in designing web pages with all allied elements like forms, text, script, graphic and so on.For example, Fontello, Adobe Edge Inspect, Foundation 3, Brackets.

12. Quality Assurance Tools:

- Quality assurance in a software organization is monitoring the engineering process and methods adopted to develop the software product in order to ensure conformance of quality as per organization standards. For example, SoapTest, AppsWatch, JMeter.

13. Maintenance Tools:

- Software maintenance includes modifications in the software product after it is delivered. For example, Bugzilla for defect tracking, HP Quality Center.

2.9. Role of CASE Tools in Modern Software Engineering Practices

1. **Automation of Repetitive Tasks**

- CASE tools automate routine tasks such as code generation, documentation, and testing. This reduces manual effort, minimizes errors, and speeds up development.

Example: UML tools like StarUML generate code skeletons from design diagrams.

2. Improved Productivity

- By automating tasks and providing integrated environments, CASE tools help developers and teams complete projects faster with fewer resources.

Example: Integrated Development Environments (IDEs) like Visual Studio combine editing, compiling, and debugging in a single tool.

3. Enhanced Accuracy and Consistency

- CASE tools enforce consistency across design, coding, and documentation through standard templates and auto-validation features.

Example: Modeling tools ensure that all diagrams conform to UML standards.

4. Better Documentation and Traceability

- Automatic generation of documentation ensures that system design, requirements, and code are well documented, facilitating future maintenance and auditing.

Example: Tools like Doxygen generate API documentation from annotated code.

5. Support for Agile and DevOps Practices

- Modern CASE tools integrate seamlessly with agile methodologies and DevOps pipelines, supporting continuous integration and delivery (CI/CD).

Example: Tools like Jenkins, Jira, and GitHub Actions automate builds, tests, and deployment processes.

6. Collaborative Development

- CASE tools provide a platform for multiple developers to collaborate in real-time, share designs, code, and track progress.

Example: Tools like Jira and Confluence integrate project management with documentation and issue tracking.

7. Quality Assurance and Testing

- Testing tools under the CASE umbrella help ensure software quality by automating unit, integration, and regression testing.

Example: Selenium automates browser testing, improving web application reliability.

8. Effective Project Management

- CASE tools assist in planning, tracking, resource management, and reporting, ensuring project goals are met on time and within budget.

Example: Microsoft Project and Trello support sprint planning and progress tracking.

9. Reusability and Maintainability

- CASE tools promote modular design and documentation, enhancing the reusability and maintainability of code components.

Example: Repository-based tools allow reuse of code libraries and design patterns.

10. Support for Standards and Compliance

- CASE tools help organizations comply with industry standards and best practices by enforcing development methodologies and generating compliance reports.

Example: Tools like Enterprise Architect support standards like TOGAF, BPMN, and ISO 9001.